

提高级 CSP-S 第 16 套初赛模拟试题

一、单项选择题(共 15 题,每题 2 分,共计 30 分;每题有且仅有一个正确选项)

- 请选出以下最小的数()。
A. $(2333)_{10}$ B. $(1926)_{16}$ C. $(817)_9$ D. $(10010110)_2$
- 5 在模 7 意义下的逆元是()。
A. 1 B. 2 C. 3 D. 4
- 今年(2022)是第()届 NOI。
A. 37 B. 38 C. 39 D. 40
- DLX 解决 $n * m$ 矩阵,有 k 个 1 精确覆盖的时间复杂度为()。
A. $O(n^m)$ B. $O(m^n)$
C. $O(2^n)$ D. $O(c^k)$, c 是接近 1 的常数
- 现有苹果,梨子,橘子,桃子无数个,现要挑出 n 个组成果篮,要求苹果一定要是偶数个,梨子是 3 的倍数,橘子至多一个,桃子至多两个。则方案数为()。
A. $n+1$ B. $2n+1$ C. $\frac{1}{2}n(n+1)$ D. $\frac{n+1}{2}$
- 现有 n 个点的完全图,对每条边染成黑白两色之一,则 n 最小为()时,对于任意染色,这个图中都一定有一个点数为 3 的团,满足边都是黑色,或者有一个点数为 3 的团,满足边都是白色。
A. 4 B. 5 C. 6 D. 7
- 一个递归算法的时间复杂度可以用某个函数 $T(n)$ 表示,且满足 $T(n) = T(n/2) + n \log n$,则 $T(n)$ 为()。
A. $O(n^2)$ B. $O(n \sqrt{n})$ C. $O(n \log n)$ D. $O(n \log^2 n)$
- 有如下程序:
f(n, m):
 if n==0 return 0
 if isprime(n) return f(n-1, m) + m/n
 else return f(n-1, m)
其中 isprime(n) 当 n 是质数时返回 1,否则返回 0。则 $f(20, 20)$ 的返回值为()。
A. 100 B. 101 C. 25 D. 26
- 二维最小圆覆盖的最优时间复杂度是()。
A. $O(n^2)$ B. $O(n^3)$ C. $O(n \log n)$ D. $O(n)$
- 定义函数 $C(l, r) = \sum_{i=1}^r \sum_{j=1}^r [\gcd(i, j) \geq l]$, 定义 $f(n, k)$ 表示对于所有长度为 $k+1$ 的整数序列 $l_0, l_1, \dots, l_k$, 满足 $0 = l_0 < l_1 < \dots < l_k = n$, 最小的 $\sum_{l_0 \leq i \leq k} C(l_{i-1}+1, l_i)$, 则 $f(10^9, 10^8)$ 的值为()。
A. 1145141919810 B. 1926081719260817 C. $2 * 10^9$ D. 10^9
- 将一条一端点在原点,另一端点在 x 正半轴上的一条长度为 1 的线段以原点为端点逆时针随机旋转 $\theta \in [l, r]$, $0 < l < r < \frac{\pi}{2}$, 则另一端点到 x 轴的期望距离为()。
A. $\cos l - \cos r$ B. $\sin r - \sin l$ C. $\frac{\cos l - \cos r}{r-l}$ D. $\frac{\sin r - \sin l}{r-l}$
- 在 $[0, 1]$ 范围内随机选择两个端点作为线段的两 endpoint, 则线段长度的期望值为()。
A. $\frac{1}{2}$ B. $\frac{2}{5}$ C. $\frac{1}{3}$ D. $\frac{3}{10}$
- 在 $2n$ 个数中找到最大值和最小值,所需要最小比较次数为()。
A. $2n-2$ B. $3n-2$ C. $4n-1$ D. $3n-3$

14. 假设你的程序开了 $1000 * 1000$ 个长度为 1000 的 bitset, 则所用的空间约为()。
- A. 64MB B. 128MB C. 256MB D. 512MB
15. 历史上第一个书写计算机程序的人是()。
- A. 阿兰·图灵 B. 冯·诺依曼 C. 阿达·洛夫莱斯 D. 克劳德·香农

二、阅读程序(程序输入不超过数组或字符串定义的范围;判断题正确的填“√”,错误的填“×”,除特殊说明外,判断题 1.5 分,单选题 3 分,共计 40 分)

1.

```
01  #include<bits/stdc++.h>
02  using namespace std;
03  const int N=1e5+5;
04  int n, ans=0, maxn;
05  int A[N];
06  int main() {
07      int i, j, h;
08      scanf ("%d", &n);
09      for (i=1; i<=n; i++) {
10          scanf ("%d", &A[i]);
11      }
12      for (i=1; i<=n; i++) {
13          for (j=i; j<=n; j++) {
14              reverse (A+i, A+j+1);
15              maxn=0;
16              for (h=1; h<n; h++) {
17                  maxn=max (maxn, A[h]+A[h+1]);
18              }
19              ans=max (ans, maxn);
20              reverse (A+i, A+j+1);
21          }
22      }
23      printf ("%d\n", ans);
24  }
```

除特殊声明外, n 是小于等于 10^5 且大于 1 的正整数, A_i 均为 10^8 以内的正整数。完成下面的判断题和单选题:

●判断题

- (1) 答案可能会大于 $2 * 10^8$ 。()
- (2) 如果数据范围不会导致数组越界,则输出的一定是一个正整数。()
- (3) 这个程序的时间复杂度是 $O(n^3)$ 。()
- (4) 这个程序在普通计算机上无法在 1s 内跑完 $n=10^5$ 的数据。()

●单选题

- (5) 如果输入数据满足 $n=4, A_i=i$, 则输出为()。
- A. 5 B. 6 C. 7 D. 8
- (6) 如果输入数据满足 $n=100, A_i=-i^2+3i+10^7$, 则答案为()。
- A. $2 * 10^7+1$ B. $2 * 10^7+2$ C. $2 * 10^7+3$ D. $2 * 10^7+4$

2.

```
01  #include<bits/stdc++.h>
02  using namespace std;
03  const int N=1e5+5, K=150+5;
04  int n, m, k, x, y;
05  int a[N];
```

```

06 long long ans, f[K][N];
07 int main() {
08     int i, j;
09     scanf ("%d", &n);
10     for (i=1; i<=n; i++) {
11         scanf ("%d", &a[i]);
12     }
13     scanf ("%d", &m);
14     k= min(sqrt(n), 150.0);
15     for (i=1; i<=k; i++) {
16         for (j=n; j; j--) {
17             f[i][j]=a[j]+(j+i<=n ? f[i][j+i] : 0);
18         }
19     }
20     for (i=1; i<=m; i++) {
21         scanf ("%d%d", &x, &y);
22         ans=0;
23         if (y>k) for (j=x; j<=n; j+=y) {
24             ans+=a[j];
25         }
26         else {
27             ans=f[y][x];
28         }
29         printf ("%lld\n", ans);
30     }
31 }

```

除特殊声明外, n, m 是小于等于 10^5 的正整数, A_i 均为 10^9 以内的正整数。完成下面的判断题和单选题:

●判断题

- (1) 答案一定小于 10^{14} 。()
- (2) 若将第 14 行换成 $k=\text{sqrt}(n)$, 则可能会导致数组越界。()
- (3) 若将第 14 行换成 $k=\text{sqrt}(n)$ 并更改有关代码, 则这个程序在 128MB 的空间限制下会 MLE。()
- (4) 将第 14 行的 150.0 改成 100.0, 输出可能会变化。()

●单选题

- (5) 通过改变 k 的取值, 这个程序可以做到的最优时间复杂度为()。
 A. $O(n\sqrt{m}+m)$ B. $O((n+m)\sqrt{n})$ C. $O(m\sqrt{n}+n)$ D. $O(n\sqrt{n}+m)$
- (6) (4 分) 在最优时间复杂度下, k 的取值是()。
 A. $\sqrt{m}$ B. $\sqrt{n}$ C. $\frac{n}{\sqrt{m}}$ D. $\frac{m}{\sqrt{n}}$

3.

```

01 #include<bits/stdc++.h>
02 using namespace std;
03 const int N=2e5 + 5;
04 int n, R, Ans;
05 int A[N], B[N];

```

```

06 int main() {
07     // freopen("1.in", "r", stdin);
08     int i, j;
09     scanf ("%d", &n);
10     for (i=1; i<=n; i++) {
11         scanf ("%d", &A[i]);
12     }
13     sort (A+1, A+n+1);
14     for (i=n; i; i--) {
15         if (A[i]<=Ans) {
16             break;
17         }
18         if (A[i]==A[i+1]) {
19             continue;
20         }
21         for (j=1; j<=n; j++) {
22             B[j]= A[j]%A[i];
23         }
24         sort (B+1, B+n+1);
25         Ans=max (Ans, (B[n]+B[n-1])% A[i]);
26         R = 1;
27         for (j=n; j>1; j--) {
28             while (R<n&&B[R + 1]+B[j]<A[i]) {
29                 R++;
30             }
31             if (R==j) {
32                 R--;
33             }
34             R^1&&(Ans=max (Ans, (B[R]+B[j])) );
35         }
36     }
37     printf ("%d\n", Ans);
38 }

```

除特殊声明外, n 是大于 2 小于等于 2×10^5 的正整数, A_i 均为 10^8 以内的正整数。完成下面的判断题和单选题:

●判断题

- (1) 存在一组数据使得这个程序运算次数达到 10^9 级别。()
- (2) 若将第 16 行换成 continue, 则对程序时间复杂度没有影响。()
- (3) 这个程序可以通过一些技巧做到更优理论复杂度。()

●单选题

- (4) (4 分) 去掉第 15 至 17 行, 该程序时间复杂度是()。
 A. $O(n \log n)$ B. $O(n^2 \log n)$ C. $O(n^2)$ D. $O(n \log n \log V)$
- (5) (3.5 分) 去掉第 18 至 20 行, 该程序时间复杂度是()。
 A. $O(n \log n)$ B. $O(n^2 \log n)$ C. $O(n^2 \log V)$ D. $O(n \log n \log V)$
- (6) (4 分) 该程序时间复杂度是()。
 A. $O(n \log n)$ B. $O(n^2 \log n)$ C. $O(n \log n \log V)$ D. $O(\frac{n \log n \log V}{\log \log V})$

三、完善程序(单选题,每小题 3 分,共计 30 分)

1. Reverse

给定一个长度为 n 的数组 a 和一个包含 m 个正整数的集合 b , 保证对于所有的 $1 \leq i \leq m$, 都有 $1 \leq 2b_i \leq n$ 。你可以对 a 数组执行若干次操作, 每次操作中, 你可以: 选出一个在 b 集中出现过的数 x , 选择 a 数组中一个长度为 x 的子段, 然后将该子段里的所有元素的值变为其相反数。形式化地说, 选出满足 $1 \leq l \leq r \leq n$ 且 $r-l+1=x$ 的两个数 l, r , 然后对于所有的 $l \leq i \leq r$, 将 a_i 替换为 $-a_i$ 。

你希望最大化 a 序列的总和, 求出这个最大值。

试补全下列程序:

```

01  #include<bits/stdc++.h>
02  using namespace std;
03  const int N=1e6+5;
04  int T, n, m, k, x, W[N+5], A[N+5], Step, F[N+5];
05  long long ans, pus;
06  int main() {
07      int i;
08      scanf ("%d%d", &n, &m);
09      for (i=1; i<=n; i++) {
10          scanf ("%d", &A[i]);
11      }
12      Step=0;
13      while (m--) {
14          scanf ("%d", &x);
15          Step=(!Step ? x: ① );
16      }
17      memset (F, 0x3f, sizeof F);
18      for (i=1; i<=n; i++) {
19          F[i%Step]=min (F[i%Step], abs (A[i]));
20          W[i%Step]+=(A[i]<0);
21      }
22      ans=-1e18;
23      pus=0;
24      for (i=1; i<=n; i++) {
25          pus+=abs (A[i]);
26      }
27      for (i=0; i<Step; i++) {
28          if ( ② ) {
29              ③ ;
30          }
31      }
32      ans=max (ans, pus);
33      pus=0;
34      for (i=1; i<=n; i++) {
35          pus+= ④ ;
36      }
37      for (i=0; i<Step; i++) {
38          if (W[i]%2) {

```

```

39         pus-=2*F[i];
40     }
41 }
42     ans=max(ans, pus);
43     printf ("%lld\n", ans);
44 }

```

(1) ①处应填()。

A. min(Step,x) B. max(Step,x) C. Step/x D. _gcd(x,Step)

(2) ②处应填()。

A. W[i]%2==0 B. W[i]>=Step
C. W[i]<=Step D. W[i]%2==1

(3) ③处应填()。

A. pus-=F[i] B. pus-=2 * F[i] C. pus+=F[i] D. pus+=2 * F[i]

(4) ④处应填()。

A. -abs(A[i]) B. abs(A[i]) C. A[i] D. -A[i]

2. Mex

给定一棵有 n 个点的树,每个节点有点权。所有的点权构成了一个 0 到 $n-1$ 的排列。有 q 次操作,每次操作 1 为交换两个点的点权,操作 2 为查询 Mex(l) 值最大的 Mex(l) 值,其中 l 是树上的一条路径。定义一条路径 l 的 Mex 值 Mex(l) 为这条路径上最小的没有出现过的自然数。

一行是一个整数 n 代表点数;下面一行是 n 个数字,代表每个点的点权,保证是一个 0 到 $n-1$ 的排列;下面一行有 $n-1$ 个数字,第 i 个数字代表钦定 1 为根时第 $i+1$ 个点的父节点;下面一行是一个整数 q ,代表操作个数;下面 q 行,每行一组操作。

如果该行第一个数字为 1,则后面有两个数字 x,y ,代表交换 x,y 的点权,否则该行有且仅有一个数字 2,代表一次查询。

提示:考虑线段树上以值域为下标建立线段树,每个节点维护子树内的点在树上的路径,每次查询只需在线段树上二分出一段前缀在线段树上是一条路径的即可。

代码中 Ques 结构体存储了一条路径。代码中使用 st 表做到 $O(n \log n) - O(1)$ 的 LCA。试补全下列程序:

```

001 #include<bits/stdc++.h>
002 using namespace std;
003 const int N=2e5+5, M=N<<1;
004 vector<int>S[N];
005 int n, m, P[N+5], Id[N+5], x, y, op;
006 namespace Dist {
007     int Dfn[M], lg[M], Bg[N], T[M][20], d[N], H;
008     void dfs (int x, int La) {
009         Dfn[Bg[x]=++H]=x;
010         d[x]=d[La]+1;
011         for (int i:S[x]) {
012             dfs (i, x), ①;
013         }
014     }
015     int LCA (int x, int y) {
016         x=Bg[x];
017         y=Bg[y];
018         x>y&&(swap(x, y), 0);

```

```

019         int D=lg[y-x+1];
020         return ②;
021     } int Dt (int x, int y) {
022         return d[x]+d[y]-2*d[LCA (x, y)];
023     }
024     void BD() {
025         int i, j;
026         dfs (1, 0);
027         for (i=2; i<=H; i++) {
028             lg[i]=lg[i / 2]+1;
029         }
030         for (i=H; i; i--) {
031             for (T[i][0]=Dfn[i], j=1; i+(1<<j)-1<=H; j++) {
032                 T[i][j]= ②;
033             }
034         }
035     }
036 }
037 struct Ques {
038     int x, y, Fl;
039 };
040 Ques Merge (Ques x, Ques y) {
041     if (x.Fl==-1 || y.Fl==-1) return (Ques) {
042         0, 0, -1
043     };
044     if ( ⑤ ) {
045         return y;
046     }
047     if ( ⑤ ) {
048         return x;
049     }
050     int D1=Dist::Dt (x.x, x.y), D2=Dist::Dt (x.x, y.x), D3=
Dist::Dt (x.x, y.y), D4=Dist::Dt (x.y, y.x), D5=Dist::Dt (x.y, y.y),
D6=Dist::Dt (y.x, y.y);
051     if (D1==D2+D4&&D1==D3+D5) return (Ques) {
052         x.x, x.y, 1
053     };
054     if ( ③ ) return (Ques) {
055         x.x, y.x, 1
056     };
057     if (D3==D1+D5&&D3==D2+D6) return (Ques) {
058         x.x, y.y, 1
059     };
060     if (D4==D1+D2&&D4==D5+D6) return (Ques) {
061         x.y, y.x, 1
062     };
063     if (D5==D1+D3&&D5==D4+D6) return (Ques) {

```

```

064     x.y, y.y, 1
065 };
066 if (D6==D2+D3&&D6==D4+D5) return (Ques) {
067     y.x, y.y, 1
068 };
069 return (Ques) {
070     0, 0, -1
071 };
072 }
073 namespace Tree {
074 #define ls v<<1
075 #define rs v<<1|1
076     Ques F[M<<1];
077     void Up (int v) {
078         F[v]=Merge (F[ls], F[rs]);
079     } void BD (int l=0, int r=n-1, int v=1) {
080         if (l==r) {
081             F[v]=(Ques) {
082                 Id[l], Id[l], 1
083             };
084             return;
085         }
086         int m=l+r>>1;
087         BD (l, m, ls);
088         BD (m+1, r, rs);
089         Up (v);
090     }
091     void Ins (int x, int l=0, int r=n-1, int v=1) {
092         if (l==r) {
093             F[v]=(Ques) {
094                 Id[x], Id[x], 1
095             };
096             return;
097         }
098         int m=l+r>>1;
099         x<=m ? Ins (x, l, m, ls) : Ins (x, m+1, r, rs);
100         Up (v);
101     }
102     int Find (Ques x, int l=0, int r=n-1, int v=1) {
103         if (l==r) {
104             return ④;
105         }
106         int m=l+r >> 1;
107         return ~Merge (x, F[ls]).Fl ? Find (Merge (x, F[ls]), m+1,
            r, rs) : Find (x, l, m, ls);
108     }
109 #undef ls

```

```

110 #undef rs
111 }
112 int main() {
113     int i, j;
114     scanf ("%d", &n);
115     for (i=1; i<=n; i++) {
116         scanf ("%d", &P[i]), Id[P[i]]=i;
117     }
118     for (i=2; i<=n; i++) {
119         scanf ("%d", &x), S[x].push_back (i);
120     }
121     Dist::BD();
122     Tree::BD();
123     scanf ("%d", &m);
124     while (m--) {
125         scanf ("%d", &op);
126         if (op==2) printf ("%d\n", Tree::Find ( ⑥ ));
127         else {
128             scanf ("%d%d", &x, &y), swap (P[x], P[y]), swap (Id[P
                [x]], Id[P[y]]), Tree::Ins (P[x]), Tree::Ins (P[y]);
129         }
130     }
131 }

```

(1) ①处应填()。

- A. Dfn[++H] = x B. 1 C. Bg[x] = H D. T[i][0] = x

(2) ②处应分别填()。

- A. $T[x][D] < T[y-(1 \ll D)+1][D] ? T[x][D] : T[y-(1 \ll D)+1][D]$ 与 $(T[i][j-1] < T[i+(1 \ll j-1)][j-1] ? T[i][j-1] : T[i+(1 \ll j-1)][j-1])$
 B. $d[T[x][D]] < d[T[y-(1 \ll D)+1][D]] ? T[x][D] : T[y-(1 \ll D)+1][D]$ 与 $(T[i][j-1] < T[i+(1 \ll j-1)][j-1] ? T[i][j-1] : T[i+(1 \ll j-1)][j-1])$
 C. $T[x][D] < T[y-(1 \ll D)+1][D] ? T[x][D] : T[y-(1 \ll D)+1][D]$ 与 $(d[T[i][j-1]] < d[T[i+(1 \ll j-1)][j-1]] ? T[i][j-1] : T[i+(1 \ll j-1)][j-1])$
 D. $d[T[x][D]] < d[T[y-(1 \ll D)+1][D]] ? T[x][D] : T[y-(1 \ll D)+1][D]$ 与 $(d[T[i][j-1]] < d[T[i+(1 \ll j-1)][j-1]] ? T[i][j-1] : T[i+(1 \ll j-1)][j-1])$

(3) ③处应填()。

- A. $D2 == D3 + D6 \ \&\& \ D2 == D4 + D5$ B. $D2 == D3 + D5 \ \&\& \ D6 == D1 + D4$
 C. $D2 == D5 + D6 \ \&\& \ D6 == D1 + D4$ D. $D2 == D3 + D6 \ \&\& \ D2 == D1 + D4$

(4) ④处应填()。

- A. 1 B. l+1
 C. $x = \text{Merge}(x, F[v]), \sim x.Fl ? l+1 : l$
 D. $\sim x.Fl ? l+1 : l$

(5) ⑤处应填()。

- A. !x.Fl 与 !y.Fl B. !y.Fl 与 !x.Fl C. x.Fl 与 y.Fl D. y.Fl 与 x.Fl

(6) ⑥处应填()。

- A. (Ques){0,0,-1} B. (Ques){1,1,1} C. (Ques){0,0,0} D. (Ques){0,0,1}